Lecture 19 : Project Estimation Techniques (Contd.)

Introduction

In software project management, accurately estimating project size is a foundational step for planning resources, timelines, and budgets. This lecture delves into parametric models, a sophisticated approach to size estimation that moves beyond the limitations of traditional methods like Lines of Code (LOC). We specifically focus on the Albrecht/IFPUG Function Point model, a widely recognized technique that quantifies software functionality from a user's perspective, offering a more robust and language-independent measure of system size. Understanding these models is crucial for students and professionals aiming to improve the predictability and control of software development projects.

Core Concepts

Term	Definition	Significance
`Parametric Model`	An estimation technique that uses mathematical formulas based on project parameters (like size) to predict effort or cost.	Provides a structured, quantifiable approach to estimation, reducing subjectivity.
`Function Points (FPs)`	A unit of measure for software size based on the functionality delivered to the user, independent of programming language or technology.	Overcomes drawbacks of LOC by focusing on user-centric functionality, enabling early estimation.
`IFPUG`	International Function Points Users Group, an organization that promotes and develops consistent guidelines for function point counting.	Ensures standardization and accuracy in FP counting across different projects and organizations.
`External Input (EI)`	A transaction that processes data coming from outside the application boundary and	

Term	Definition	Significance
	updates internal computer files.	Represents user-initiated data entry that changes the system's state.
`External Output (EO)`	A transaction that extracts and displays data from internal computer files, sending it outside the application boundary, often in the form of reports.	Represents information generated by the system for the user.
`External Inquiry (EQ)`	A user-initiated transaction that provides information by retrieving data but does not update any computer files.	Represents user requests for information without altering system data.
`Logical Internal File (LIF)`	A user-identifiable group of logically related data maintained within the boundary of the application.	Corresponds to data stores managed directly by the system being developed.
`External Interface File (EIF)`	A user-identifiable group of data referenced by the application but maintained within the boundary of another application.	Represents data shared with or accessed from other independent systems.
`Unadjusted Function Point (UFP)`	The initial raw count of function points derived from the weighted sum of EIs, EOs, EQs, LIFs, and EIFs.	Provides a preliminary measure of functional size before considering technical complexity.
`Value Adjustment Factor (VAF)`	A multiplier derived from assessing 14 General System Characteristics (GSCs) that refines the UFP to account for technical and environmental complexity.	Adjusts the raw functional size to reflect the actual effort required due to project-specific complexities.

Term	Definition	Significance
`Total Degree of Influence (TDI)`	The sum of the 'Degree of Influence' ratings (0-5) for each of the 14 General System Characteristics.	Quantifies the overall technical and environmental complexity of the project.
`General System Characteristics (GSC)`	14 factors (e.g., data communication, performance, reusability) that describe the technical and environmental aspects of a software project.	Used to determine the VAF, providing a comprehensive view of project complexity beyond just functionality.

Detailed Analysis

1. Evolution from LOC to Function Points

Historically, Lines of Code (LOC) was a common metric for software size. However, LOC suffers from several significant drawbacks:
* **Rewards Verbosity**: It tends to reward verbose coding styles and penalize concise, efficient designs.
* **Lack of Standardization**: There are no universal industry standards (like ISO) for counting LOC, leading to inconsistencies.
* **Platform/Language Dependence**: LOC counts vary drastically across different programming languages, platforms, and organizations, making cross-project comparisons difficult.
* **Inapplicability to 4GLs**: Many modern 4th Generation Languages (4GLs) do not rely on traditional lines of code, rendering LOC irrelevant.
* **Misleading**: LOC can be easily manipulated or misinterpreted, leading to inaccurate estimations.

Function Points (FPs) were introduced by Albrecht at IBM to address these issues. Albrecht sought a way to measure the relative productivity of different programming languages and the size of an application without counting LOC. FPs offer several advantages:
* **Language, Tool, and Methodology Independent**: FPs measure functionality, not implementation details, making them universally applicable.
* **Early Estimation**: FPs can be estimated early in the software development lifecycle, even during analysis and design phases, when requirements are being defined. This is a significant improvement over LOC, which can only be accurately counted after coding.

* **User-Centric View**: FPs are based on the user's external view of the system, making them more understandable and relatable even for non-technical stakeholders.

The International Function Points Users Group (IFPUG) was established to promote and encourage the use of FPs and develop consistent, accurate counting guidelines. IFPUG provides benefits such as networking, counting practices manuals, research support, and certification.

2. Objectives and Timing of Function Point Counting

The primary objectives of function point counting are:
* To measure the functionality that the user requests and receives.
* To measure software development and maintenance effort independently of the underlying technology.
* To provide a simple enough process to minimize measurement overhead.
* To act as a consistent measure across various projects and organizations.

Function points should be counted "early and often" in the development process. The sooner a project's deliverables can be quantified, the better it can be controlled and monitored. Under IFPUG 4.1 guidelines, FPs can be counted once requirements are finalized. The estimate of size in FPs can be continuously refined throughout the development cycle as more information becomes available. This continuous refinement allows for tracking scope creep and breakage.

3. Steps in Function Point Computation

The computation of function points involves a structured, multi-step process:
1. **Identify Counting Scope and Application Boundary**: Clearly define what functionality is included in the system being measured and its boundaries.
2. **Count Data Functions**: Identify and count Logical Internal Files (LIFs) and External Interface Files (EIFs).
3. **Count Transactional Functions**: Identify and count External Inputs (EIs), External Outputs (EOs), and External Inquiries (EQs).
4. **Determine Unadjusted Function Point (UFP) Count**: Calculate the initial FP count using complexity multipliers for each function type.
5. **Determine Value Adjustment Factor (VAF)**: Assess 14 General System Characteristics (GSCs) to derive a factor that accounts for technical and environmental complexity.
6. **Calculate Adjusted Function Point Count**: Multiply the UFP by the VAF to get the final, adjusted FP count.

4. Key Components: External User Types

Albrecht identified five key components, also known as external user types, for function point analysis:
* **External Input (EI)**: These represent transactions that process data coming from outside the application boundary and update internal computer files. For example, an online entry that updates customer information in a customer info file.
* **External Output (EO)**: These are transactions that extract and display data from internal computer files, sending it outside the application boundary. They generally involve creating reports, such as a category summary report for an end-user.
* **External Inquiry (EQ)**: These are user-initiated actions that provide information by retrieving data but do not update any computer files. An example is a user querying customer details, where the system displays the information without altering it.
* **Logical Internal File (LIF)**: This equates roughly to a data store in system analysis terms. It is a user-identifiable group of logically related data maintained within the boundary of the application. A customer info file updated by the application itself is an LIF.
* **External Interface File (EIF)**: This represents data retrieved from a data store that is actually maintained by another different application. For instance, a zip code table used for validation but maintained by an external system would be an EIF.

5. Calculating Unadjusted Function Points (UFP)

To calculate the UFP, each of the five external user types (EI, EO, EQ, LIF, EIF) is assigned a complexity level: Low, Medium, or High. Albrecht proposed specific complexity multipliers (weights) for each type based on its complexity.

External User Type	Low Complexity	Medium Complexity	High Complexity
EI	3	4	6
EO	4	5	7
EQ	3	4	6
LIF	7	10	15
EIF	5	7	10

The UFP is calculated by summing the product of the count of each user type and its corresponding complexity multiplier.
For example, in a spell-checker application:
* 2 user inputs (document file name, personal dictionary name) - assumed average (medium) complexity.
* 3 user outputs (error report, word count, misspelled error count) - assumed average (medium) complexity.
* 2 user inquiries (# processed words?, # spelling errors?) - assumed average (medium) complexity.
* 1 internal file (dictionary) - assumed average (medium) complexity.
* 2 external files (document file, personal dictionary) - assumed average (medium) complexity.

The UFP calculation would be:
UFP = (2 EIs * 4) + (3 EOs * 5) + (2 EQs * 4) + (1 LIF * 10) + (2 EIFs * 7)
UFP = 8 + 15 + 8 + 10 + 14 = 55

6. Refining Function Points with the Value Adjustment Factor (VAF)

The UFP provides a raw measure of functional size. To account for the technical and environmental complexity of the project, a Value Adjustment Factor (VAF) is applied. This factor is derived from 14 General System Characteristics (GSCs), which are evaluated on a scale of 0 to 5 for their "Degree of Influence" (DI):
* 0: Not present, or no influence
* 1: Incidental influence
* 2: Moderate influence
* 3: Average influence
* 4: Significant influence
* 5: Strong influence throughout

The 14 GSCs include factors like Data Communication, Distributed Data Processing, Performance Objectives, Reusability, On-Line Data Entry, End-User Efficiency, and Maintainability.

The procedure to determine the VAF is:
1. **Evaluate each of the 14 GSCs**: Assign a DI value (0-5) to each characteristic.
2. **Calculate Total Degree of Influence (TDI)**: Sum the DI values for all 14 GSCs.
3. **Compute VAF**: Use Albrecht's formula:

```
VAF = (TDI * 0.01) + 0.65
```

This VAF (also sometimes called Technical Complexity Factor or TCF) expresses the overall impact of these parameters on the development effort.

Finally, the Adjusted Function Point (FP) is calculated by multiplying the UFP by the VAF:

```
FP = UFP * VAF
```

Example Calculation of VAF and Adjusted FP:
Continuing with the spell-checker example (UFP = 55):
* **Scenario 1: All 14 GSCs have a DI of 3 (average influence).**
TDI = 14 GSCs * 3 DI/GSC = 42
VAF = (42 * 0.01) + 0.65 = 0.42 + 0.65 = 1.07
Adjusted FP = 55 * 1.07 = 58.85

- **Scenario 2: GSCs have varying DIs (e.g., some 0, some 3, some 5).**
  Suppose the sum of DIs for all 14 GSCs results in a TDI of 36.
  VAF = (36 * 0.01) + 0.65 = 0.36 + 0.65 = 1.01
  Adjusted FP = 55 * 1.01 = 55.55

This adjusted FP provides a more realistic estimate of the software's size, considering both its functional requirements and its technical complexities.

Key Takeaways

- [ ] Lines of Code (LOC) has significant drawbacks as a size estimation metric due to its dependence on language, design style, and lack of standardization.
- [ ] Function Points (FPs) offer a language-independent, user-centric measure of software size, overcoming LOC limitations.
- [ ] Albrecht/IFPUG Function Point analysis is a top-down approach that can be used early in the software development lifecycle.
- [ ] The five key components for FP analysis are External Inputs (EI), External Outputs (EO), External Inquiries (EQ), Logical Internal Files (LIF), and External Interface Files (EIF).
- [ ] Unadjusted Function Points (UFP) are calculated by summing the product of the count of each component type and its complexity multiplier (Low, Medium, or High).
- [ ] The Value Adjustment Factor (VAF) refines the UFP by considering 14 General System Characteristics (GSCs) and their Degree of Influence (DI).
- [ ] The Total Degree of Influence (TDI) is the sum of DIs for all 14 GSCs, used in the VAF formula: `VAF = (TDI * 0.01) + 0.65`.
- [ ] The final Adjusted Function Point (FP) is obtained by multiplying the UFP by the VAF, providing a more accurate size estimate.

- [ ] Function points should be continuously recounted and refined throughout the development process to track scope changes.

Conclusion

The Albrecht/IFPUG Function Point model provides a powerful and standardized method for estimating software size, moving beyond the inherent limitations of traditional metrics like Lines of Code. By focusing on the functionality delivered to the end-user and systematically accounting for technical and environmental complexities, this parametric model enables project managers to achieve more accurate and reliable size estimates. This precision, in turn, leads to better planning, resource allocation, and overall project control. Mastering function point analysis is an essential skill for anyone involved in software project management, as it lays the groundwork for more predictable and successful software delivery in a diverse technological landscape.